

Pieter Ayal

pietera.mullock@gmail.com | linkedin.com/in/pieterayal | github.com/pieter124

EDUCATION

University of Manchester

Manchester, United Kingdom

Bachelor of Science in Computer Science

Sep. 2024 – Jun. 2028

- Academic Performance: First Year: **75.3%** (First Class) | Second Year: **76.8%** (First Class)
- Relevant Coursework: Data Structures and Algorithms, Compilers, System Architecture, Concurrent Programming, Distributed Systems, Operating Systems, Software Engineering

EXPERIENCE

SLB

Abingdon, United Kingdom

HPC Software Engineer Intern

Sep. 2026 – Jun. 2027

Thought Machine

London, United Kingdom

Software Engineer Intern

Jun. 2026 – Sep. 2026

- Architecting and developing a full-stack **Balances Visualiser** using **TypeScript** and **React** to accelerate incident debugging for on-call engineers and simplify onboarding onto the ledger's complex accounting model.
- Owning the project end-to-end, driving the lifecycle from initial design to implementation via RFC proposals, cross-team code reviews, and technical evaluations.

EPCC

Edinburgh, United Kingdom

HPC Summer School Student

Jun. 2025

- Developed and deployed **parallel C applications** to **Cirrus** (a multi-node HPC cluster) via **SLURM**, leveraging the **Message Passing Interface (MPI)** for inter-process communication across nodes and **OpenMP** for shared-memory parallelism within nodes to solve large-scale numerical simulations.
- Analysed program performance, identifying and resolving issues related to **lock contention** and **inefficient data distribution**, reducing parallel runtime by $\sim 20\%$.

PROJECTS

Concurrent Bank Ledger | *Go, gRPC, Protobuf* | github.com/pieter124/concurrentbankledger

- Built a thread-safe in-memory banking engine in **Go** exposing **gRPC/Protobuf** endpoints for account initialisation and double-entry transfers, implementing and benchmarking **three concurrency strategies** on the same domain.
- Compared **fine-grained per-account mutexes** (deterministic lock ordering to prevent deadlock) against a **single-actor model** serialising all mutations through one goroutine; then designed a **sharded-actor architecture** partitioning accounts across N lock-free actors, with a two-phase **reserve/credit/finalize** protocol (and compensating refunds) for cross-shard transfers.
- Benchmarked all three under low and high contention, cutting per-transfer latency $\sim 2\times$ vs. the single actor and $\sim 25\%$ vs. fine-grained mutexes ($\sim 1,090$ ns/op), by sharding work across N actor goroutines instead of one.
- Added a **write-ahead log** (append + **fsync**, JSON records, replay-on-startup) giving **crash recovery** where state is rebuilt from disk after a kill/restart, plus an **idempotency layer** and race-detector-verified zero-sum conservation under concurrent load.

SAT Solver | *Rust, Cargo* | github.com/pieter124/rustSAT

- Engineered a high-performance **DPLL SAT Solver** in **Rust**, leveraging zero-cost abstractions for memory safety and execution speed to win **2nd place** in a SAT Solver Competition.
- Implemented **smart branching heuristics** (variable frequency/polarity) and **preprocessing pipelines** (Pure Literal Elimination, Tautology Removal) to aggressively prune the search space.

TECHNICAL SKILLS

Languages: Go, Python, C++, C, Rust, TypeScript

Parallelism & Infrastructure: OpenMP, MPI, SLURM, Linux, gRPC, Protobuf, Bash, Git, CMake

Competitions: Hackabot 2026 (Quansar Track) – **2nd Place**